από τους κώδικες σε αυτό το βιβλίο. Ωστόσο, για να είναι οι κώδικες μεταγλωττίσιμοι, θα πρέπει να προσθέτετε εσείς τη δήλωση package συνοδευόμενη από το όνομα του δικού σας πακέτου.

```
package gr.ihu.cs.lmous.OOJ.k2.v1;

public class ToPrinter {

    static void helloWorld() {
        System.out.println("HelloWorld!");
    }

    public static void main(String[] args) {
        helloWorld();
    }
}
```

***Κώδικας 2.2**: Η πρώτη έκδοση της κλάσης ToPrinter*

Αν τρέξουμε την κλάση, θα λάβουμε το ίδιο αποτέλεσμα με την εκτέλεση της κλάσης HelloWorld. Όπως βλέπουμε στον κώδικα 2.2, η main καλεί την helloWorld() η οποία με τη σειρά της καλεί την System.out.println που τυπώνει τη σειρά "HelloWorld!". Οι συναρτήσεις είναι υποπρογράμματα ή αλλιώς ρουτίνες και αποτελούν θεμέλιο λίθο τόσο του διαδικαστικού όσο και του αντικειμενοστρεφούς μοντέλου και θα τις χρησιμοποιούμε συστηματικά κατά την ανάπτυξη των κλάσεων. Βεβαίως, η συνάρτηση helloWorld αποτελεί την απλούστερη μορφή συνάρτησης. Είναι μια στατική void συνάρτηση χωρίς παραμέτρους. Εκτεταμένα για τις συναρτήσεις μιλάμε στην ενότητα 7. Για την ώρα, μας αρκεί αυτή η απλή μορφή.

Αν υποθέσουμε τώρα πως θέλουμε να εκτυπώσουμε τη διεύθυνσή μας χρησιμοποιώντας την κλάση ToPrinter, μπορούμε να προσθέσουμε μια κατάλληλη συνάρτηση και να την καλέσουμε στην κλάση όπως δείχνει ο κώδικας 2.3.

```
public class ToPrinter {

    static void helloWorld() {
        System.out.println("HelloWorld!");
    }

    static void address() {
        System.out.println("Ιωάννης Νάκος");
        System.out.println("Εσπερίδων 75");
        System.out.println("Καλλιθέα 65303");
        System.out.println("Αθήνα, Ελλάδα");
    }

    public static void main(String[] args) {
        helloWorld();
        address();
    }
}
```

***Κώδικας 2.3**: Η δεύτερη έκδοση της κλάσης ToPrinter*

Προσέξτε όμως ότι θα πρέπει να τοποθετήσετε τη δεύτερη έκδοση της ToPrinter σε διαφορετικό πακέτο από την πρώτη ή διαφορετικά να αλλάξετε την ονομασία της κλάσης. Τρέχοντας τον κώδικα 2.3, διαπιστώνουμε πως τυπώνεται πρώτα η σειρά "HelloWorld" και στη συνέχεια η διεύθυνση. Ένας εύκολος τρόπος για να τυπωθεί μόνο η διεύθυνση είναι να σχολιάσουμε την κλήση της HelloWorld, δηλαδή να μετατρέψουμε την main όπως δείχνει ο κώδικας 2.4.

```
    public static void main(String[] args) {
        //helloWorld();
        address();
    }
```

***Κώδικας 2.4**: Σχόλια γραμμής*

Όπως βλέπουμε στον κώδικα 2.4, από την κλήση helloWorld() έχουν προηγηθεί δύο ανάστροφοι κάθετοι (backslashes). Με αυτόν τον τρόπο πληροφορούμε τον μεταγλωττιστή πως αυτή η γραμμή είναι γραμμή σχολίων και όχι κώδικα. Τα σχόλια αγνοούνται από τον μεταγλωττιστή. Έτσι, στον bytecode που παράγει ο μεταγλωττιστής δεν συμπεριλαμβάνεται η κλήση helloWorld. Αν εκτελέσουμε τώρα την κλάση ToPrinter, θα διαπιστώσουμε ότι τυπώνει μόνο τη διεύθυνση.

Τα σχόλια είναι γενικά απαραίτητα στον κώδικά μας, όχι μόνο για να αποκλείουμε προσωρινά την κλήση συναρτήσεων αλλά κυρίως για να τοποθετούμε επεξηγήσεις μέσα στον κώδικα ώστε να γίνεται εύκολα κατανοητός από εμάς σε μεταγενέστερο χρόνο αλλά και από συνεργάτες μας που ενδέχεται να δουλέψουν με τον κώδικά μας. Επιπλέον, όπως θα δούμε στην ενότητα 8.6, τα σχόλια αξιοποιούνται και για την παραγωγή της τεκμηρίωσης των κλάσεων. Σε περίπτωση που θέλουμε να εισάγουμε σχόλια που καταλαμβάνουν περισσότερες από μία γραμμές, μπορούμε να τα εσωκλείσουμε μεταξύ των χαρακτήρων ανάστροφη κάθετος και αστερίσκος στην αρχή και αστερίσκος και ανάστροφη κάθετος στο τέλος όπως φαίνεται στον κώδικα 2.5.

```
/*
Στατική συνάρτηση χωρίς παραμέτρους
Τιμή επιστροφής: void
Τυπώνει συγκεκριμένη διεύθυνση
*/
    static void address() {
        System.out.println("Ιωάννης Νάκος");
        System.out.println("Εσπερίδων 75");
        System.out.println("Καλλιθέα 65303");
        System.out.println("Αθήνα, Ελλάδα");
    }
```

***Κώδικας 2.5**: Σχόλια πολλαπλών γραμμών*

Θα προσέξατε πως ο προσδιοριστής προσπέλασης public λείπει από τις δηλώσεις των συναρτήσεων helloWorld και address. Όταν στη δήλωση μιας συνάρτησης δεν τοποθετείται κανένας προσδιοριστής προσπέλασης, η συνάρτηση διατηρεί την εξ' ορισμού προσπέλαση (default access). Η εξ' ορισμού προσπέλαση ονομάζεται και προσπέλαση πακέτου (package access) ή και ιδιωτική προσπέλαση πακέτου (package-private), καθώς αυτή επιτρέπει την προσπέλαση μόνο από κλάσεις του ίδιου πακέτου. Στο πακέτο που τοποθετήσατε τη δεύτερη έκδοση της ToPrinter, δημιουργήστε μια κλάση CallFromAnotherClass. Η κλάση CallFromAnotherClass έχει δικαίωμα προσπέλασης στις συναρτήσεις της ToPrinter. Αντίθετα κλάσεις από άλλα πακέτα δεν έχουν αυτό το δικαίωμα. Για να κληθεί όμως μια συνάρτηση της ToPrinter από την CallFromAnotherClass θα πρέπει κατά την κλήση να προσδιορίσουμε πέραν του ονόματος της συνάρτησης και την κλάση στην οποία αυτή ανήκει.
Ο κώδικας 2. 6. παρουσιάζει την κλάση CallFromAnotherClass

```
public class CallFromAnotherClass {
    public static void main(String[] args) {
        ToPrinter.helloWorld();
        ToPrinter.address();
    }
}
```

***Κώδικας 2.6**: Κλήση στατικής συνάρτησης με πρόσβαση πακέτου από άλλη κλάση του ίδιου πακέτου*

Τρέξτε την CallFromAnotherClass και θα διαπιστώσετε πως θα εκτελεστούν οι συναρτήσεις address και helloWorld της κλάσης ToPrinter. Αν θέλουμε να δώσουμε δικαίωμα κλήσης αυτών των συναρτήσεων από κλάσεις έξω από το πακέτο που ανήκουν θα πρέπει να χαρακτηρίσουμε τις συναρτήσεις ως public.

2.4 Συμβάσεις ονοματολογίας

Η Java διαφοροποιεί μεταξύ πεζών και κεφαλαίων (case sensitive). Αν για παράδειγμα προσπαθήσετε να καλέσετε τη συνάρτηση helloWorld και τη γράψετε ως helloworld, η Java δεν θα την αναγνωρίσει. Για την

ονομασία των κλάσεων, των συναρτήσεων αλλά και άλλων αντικειμένων έχουν καθιερωθεί συμβάσεις ονοματολογίας με παγκόσμια ισχύ που θα πρέπει να ακολουθούνται πιστά. Ο μεταγλωττιστής δεν θα μας ενοχλήσει αν παραβούμε μια τέτοια σύμβαση. Ωστόσο, η τήρησή τους είναι αναγκαία ώστε οι κώδικές μας να είναι αναγνώσιμοι από εμάς ή από τρίτους. Πιο συγκεκριμένα, τα ονόματα των κλάσεων πρέπει να αρχίζουν με κεφαλαίο. Αντίθετα, τα ονόματα των συναρτήσεων πρέπει να αρχίζουν από πεζό. Και στις δύο περιπτώσεις, αν το όνομα είναι σύνθετο, π.χ. helloWorld, κάθε επόμενο συνθετικό πρέπει να αρχίζει από κεφαλαίο. Πιο εκτεταμένα για το θέμα αυτό μιλάμε στην ενότητα 3.4 όπου παρουσιάζουμε τους χαρακτήρες που είναι διαθέσιμοι προς χρήση για τη διαμόρφωση των ονομάτων κλάσεων, συναρτήσεων και άλλων αναγνωριστικών (identifier) που χρησιμοποιούμε στα προγράμματά μας.

2.5 Βασική Έξοδος

Είδαμε προηγουμένως πως με την System.out.println μπορούμε να τυπώσουμε μια σειρά χαρακτήρων στην οθόνη. Θα προσέξατε πως κάθε γραμμή της διεύθυνσης τυπώνεται σε διαφορετική γραμμή της οθόνης. Αυτό οφείλεται στην println, η οποία, αφού εμφανίζει τη σειρά χαρακτήρων που της έχουμε δώσει, προετοιμάζει την έξοδο έτσι ώστε αυτή να συνεχίσει σε νέα γραμμή. Οι χαρακτήρες ln στο τέλος της println αυτόν τον σκοπό εξυπηρετούν. Αν αυτό είναι κάτι που δεν θέλουμε, τότε αντί για println θα χρησιμοποιήσουμε τη συνάρτηση print.

Προσθέστε τη συνάρτηση prtABC (κώδικας 2.7) στην ToPrinter, καλέστε την στην main και τρέξτε την κλάση. Η νέα έκδοση της κλάσης φαίνεται στον κώδικα 2.7.

```
public class ToPrinter {

    static void helloWorld() {
        System.out.println("HelloWorld!");
    }

    static void address() {
        System.out.println("Ιωάννης Νάκος");
        System.out.println("Εσπερίδων 75");
        System.out.println("Καλλιθέα 65303");
        System.out.println("Αθήνα, Ελλάδα");
    }

    static void prtABC() {
        System.out.print("A");
        System.out.print("B");
        System.out.println("C");
    }

    public static void main(String[] args) {
        prtABC();
        helloWorld();
    }
}
```

***Κώδικας 2.7**: print και println*

Παρατηρήστε πως στην τελευταία γραμμή της prtABC έχουμε χρησιμοποιήσει println και όχι print. Επιλέξαμε αυτήν την προσέγγιση ώστε οποιαδήποτε έξοδος μετά την κλήση της prtABC να εμφανιστεί σε νέα γραμμή. Έτσι ο κώδικας 2.7 εμφανίζει το HelloWorld! κάτω από τη σειρά ABC. Αν δεν είχαμε βάλει println στην τελευταία print της prtABC, τότε ο κώδικας θα εμφάνιζε ABCHelloWorld!. Περισσότερα για την έξοδο και είσοδο στις ενότητες 4.2 και 4.3.

2.6 Ασκήσεις

Δημιουργήστε μια κλάση X01. Σε αυτήν προσθέστε τις συναρτήσεις:

prtRomvos: Τυπώνει έναν ρόμβο που σχηματίζεται από αστερίσκους όπως φαίνεται παρακάτω:

```
        ***
    **********
**************
**************
    **********
        ***
```

prtVellos: Τυπώνει ένα βέλος όπως φαίνεται παρακάτω:

```
                            *****
                             *****
                              *****
                               *****
********************************
********************************
********************************
                               *****
                              *****
                             *****
                            *****
```

Τρέξτε τις συναρτήσεις και διαπιστώστε πως λειτουργούν σωστά.

Βιβλιογραφία

[1] “Subroutine,” Wikipedia. Sep. 12, 2021. Accessed: Sep. 14, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Subroutine&oldid=1043969790

Κεφάλαιο 3

Σύνοψη
Σε αυτήν την ενότητα εξετάζεται διεξοδικά η έννοια της τοπικής μεταβλητής και αναλύονται σε βάθος οι θεμελιώδεις τύποι δεδομένων της Java. Περιλαμβάνεται παρουσίαση των ακολουθιών ελέγχου, των συμβάσεων ονοματολογίας, του φαινομένου υπερχείλισης μεταβλητής. Εξηγείται η έκφραση αριθμητικών σταθερών στο δυαδικό, οκταδικό και δεκαεξαδικό σύστημα αρίθμησης καθώς και ο επιστημονικός συμβολισμός τους. Επιπλέον, γίνεται εισαγωγή στον βασικό τύπο διαχείρισης αλφαριθμητικών δεδομένων.

Προαπαιτούμενη γνώση
Η ενότητα 2 αυτού του εγχειριδίου που περιλαμβάνει τις έννοιες: πακέτο (package), η main ως κύρια είσοδος στο πρόγραμμα, το υποπρόγραμμα ως void συνάρτηση χωρίς παραμέτρους, βασική έξοδος στην οθόνη.

Λέξεις κλειδιά
Τοπική μεταβλητή (local variable), τοπική σταθερά (local constant), μπλοκ κώδικα (block), εμβέλεια (scope), χρόνος ζωής μεταβλητής (variable life time), αναγνωριστικό (identifier), κυριολεκτική σταθερά (literal), αρχικοποίηση (initialization), χρόνος μεταγλώττισης (Compile time), χρόνος εκτέλεσης (Run time), δεσμευμένη λέξη (reserved word), λέξη κλειδί (keyword), θεμελιώδεις τύποι δεδομένων (Primitive Data Types), κλάση (Class), εκθετική μορφή (exponential Notation) ή επιστημονικός συμβολισμός (scientific notation), αριθμός κινητής υποδιαστολής (floating point number), ακολουθία ελέγχου (control sequence), χαρακτήρας διαφυγής (escape character), υπερχείλιση (overflow), λάθος χρόνου εκτέλεσης (run time error ή bug), λάθος μεταγλώττισης (compile error), αυτόματη μετατροπή τύπου (automatic type conversion) , αλφαριθμητική σειρά (String), σύνδεση αλφαριθμητικών σταθερών (concatenation).

3 Μεταβλητές και Θεμελιώδεις τύποι

Αναφέραμε προηγουμένως πως ένα πρόγραμμα είναι μια σειρά εντολών προς τη μηχανή. Αυτό είναι σωστό, ωστόσο πρόκειται για τη μισή αλήθεια, καθώς οι εντολές συχνά επενεργούν σε δεδομένα. Επομένως, ένα πρόγραμμα για να εκτελεστεί θα πρέπει να φορτωθούν στη μνήμη του υπολογιστή, όχι μόνο οι εντολές που το απαρτίζουν αλλά και τα δεδομένα στα οποία επενεργούν οι εντολές. Το πρόγραμμα φορτώνει τα δεδομένα σε κατάλληλες θέσεις στη μνήμη προς τις οποίες διατηρεί εύκολη πρόσβαση και από εκεί τα διαχειρίζεται. Η πρόσβαση στις θέσεις μνήμης που αντιστοιχούν σε δεδομένα γίνεται μέσω των μεταβλητών (variables).

Ωστόσο, υπάρχουν δεδομένα διάφορων τύπων. Για παράδειγμα, κάποια δεδομένα μπορεί να είναι ακέραιοι αριθμοί, κάποια άλλα μπορεί να είναι πραγματικοί αριθμοί ή χαρακτήρες. Όμως τα δεδομένα αποθηκεύονται στη μνήμη ως σειρές από bits. Επομένως, το πρόγραμμα θα πρέπει να γνωρίζει η κάθε σειρά από bits που αντιστοιχεί σε ορισμένο δεδομένο, τι τύπου είναι ώστε να μπορέσει να τη διαχειριστεί σωστά. Έτσι οι μεταβλητές χαρακτηρίζονται ως προς τον τύπο των δεδομένων που διαχειρίζονται, δηλαδή υπάρχουν μεταβλητές κατάλληλες για ακέραιους, μεταβλητές κατάλληλες για πραγματικούς, κοκ.

Στο κεφάλαιο αυτό παρουσιάζουμε καταρχάς τις τοπικές μεταβλητές και κάποια βασικά χαρακτηριστικά τους όπως η εμβέλεια και ο χρόνος ζωής, στη συνέχεια παρουσιάζουμε τους θεμελιώδεις τύπους της Java. Μετά εξηγούμε τη σημασία και χρήση των ακολουθιών ελέγχου και περιγράφουμε τα αποδεκτά ονόματα μεταβλητών. Ακολουθεί, η επεξήγηση της υπερχείλισης και η έκφραση αριθμητικών σταθερών σε όλα τα συστήματα αρίθμησης που υποστηρίζονται. Τέλος, παρουσιάζεται ο τύπος αλφαριθμητικής σταθεράς ο οποίος δεν είναι μεν θεμελιώδης αλλά το ίδιο βασικός και συχνά αναγκαίος όσο και οι θεμελιώδεις τύποι.

3.1 Τοπικές Μεταβλητές

Ένα πρόγραμμα χρειάζεται να διαχειριστεί κάποια δεδομένα. Η διαχείριση των δεδομένων επιτυγχάνεται με τη βοήθεια των μεταβλητών (variables). Η Java υποστηρίζει μια ποικιλία μεταβλητών, τις τοπικές μεταβλητές

(local variables), τις παραμέτρους (parameters), τις στατικές μεταβλητές (class variables ή static fields) και τις μεταβλητές στιγμιότυπου (instance variables)[1] .

Η συζήτηση στην ενότητα αυτή θα περιοριστεί στις τοπικές μεταβλητές. Οι παράμετροι και οι στατικές μεταβλητές παρουσιάζονται στις ενότητες 7.3 και 7.4, αντίστοιχα, ενώ οι μεταβλητές στιγμιότυπου αφορούν το αντικειμενοστρεφές μοντέλο και εξετάζονται στην ενότητα 11.

Για να δηλώσουμε μια τοπική μεταβλητή[2] στην Java, πρέπει να προσδιορίσουμε τον τύπο της και να της δώσουμε ένα αναγνωριστικό όνομα. Για παράδειγμα, ο κώδικας

```
int k;
```

δηλώνει μια ακέραιη μεταβλητή με αναγνωριστικό το k.

Ας δούμε όμως μερικούς αναγκαίους ορισμούς.

Μπλοκ κώδικα: Ο κώδικας μεταξύ μιας αγκύλης που ανοίγει '{' και μιας αγκύλης που κλείνει '}' συνιστά ένα μπλοκ (block) κώδικα.

Κάθε μεταβλητή που δηλώνεται μέσα σε ένα μπλοκ κώδικα είναι τοπική σε αυτό το μπλοκ.

```
void f() {
     int k;
}
```

Στη συνάρτηση f(), έχουμε δηλώσει την τοπική μεταβλητή ακέραιου τύπου k. Ο int είναι ένας θεμελιώδης τύπος της Java. Οι θεμελιώδεις τύποι των μεταβλητών παρουσιάζονται παρακάτω στην ομώνυμη ενότητα. Για την ώρα μας ενδιαφέρουν τα ακόλουθα:

Η εκτέλεση της int k; έχει ως αποτέλεσμα να δεσμευτεί χώρος στη μνήμη που έχει εκχωρηθεί στο πρόγραμμά μας, με μέγεθος ίσο με το μέγεθος μια ακέραιας μεταβλητής. Ο χώρος αυτός συνδέεται με το αναγνωριστικό (identifier) k. Η μεταβλητή k είναι τοπική στο μπλοκ που ορίζει την f. Κάθε φορά που μέσα στην f αναφερόμαστε στην k, στην ουσία προσπελαύνουμε τη συγκεκριμένη μνήμη. Ωστόσο, δεν έχουμε δώσει κάποια αρχική τιμή στην k. Με άλλα λόγια, η k είναι αναρχικοποίητη (unitialized)[3]. Προκειμένου να μας προστατεύσει από πιθανά λάθη που προκύπτουν από τη χρήση αναρχικοποίητων μεταβλητών, η Java δεν μας επιτρέπει να την προσπελάσουμε παρά μόνο για να την αρχικοποιήσουμε[4]. Έτσι ο παρακάτω κώδικας παράγει ένα λάθος μεταγλώττισης (Compile Error).

```
void f() {
     int k;
     System.out.println(k);
}
```

Αντίθετα, ο κώδικας

```
void f() {
     int k;
```

---

[1] *Επιπλέον, οι μεταβλητές ανάλογα με τον τύπο τους μπορεί να είναι μεταβλητές αναφοράς (reference variable) ή **μεταβλητές τιμής** (value variable).*

[2] *Πριν τον τύπο μπορεί να τοποθετηθούν τροποποιητές (modifiers). Στους πιθανούς τροποποιητές περιλαμβάνονται οι **προσδιοριστές προσπέλασης** (access specifer), η δεσμευμένη λέξη **static** και η δεσμευμένη λέξη **final**. Από αυτούς τους τροποποιητές μόνο ο **final** συντάσσεται με δήλωση τοπικών μεταβλητών.*

[3] *Η Java δεν αρχικοποιεί αυτόματα τις τοπικές μεταβλητές. Δεν ισχύει το ίδιο για τους υπόλοιπους τύπους μεταβλητών.*

[4] *Μπορούμε ωστόσο να προσπελάσουμε μια μεταβλητή που ενδεχομένως είναι αναρχικοποίητη εφόσον την περικλείσουμε σε try {} catch μπλοκ. Περισσότερα επ'αυτού κατά τη μελέτη των εξαιρέσεων, ενότητα ?*

```
        k = 0;
        System.out.println(k);
}
```

περνάει μεταγλώττιση και όταν εκτελεστεί τυπώνει στην οθόνη το περιεχόμενο της k, δηλαδή το 0.

Εμβέλεια: Εμβέλεια (Scope) μιας μεταβλητής ονομάζεται η περιοχή εκείνη του κώδικα που η μεταβλητή είναι αναγνωρίσιμη από τον μεταγλωττιστή.

Η εμβέλεια των τοπικών μεταβλητών αρχίζει από τη γραμμή στην οποία δηλώθηκαν και επεκτείνεται σε όλο το μπλοκ μέσα στο οποίο δηλώθηκαν. Στον κώδικα που ακολουθεί, ο μεταγλωττιστής θα παράξει λάθος καθώς προσπαθούμε να προσπελάσουμε τη μεταβλητή i έξω από το μπλοκ στο οποίο δηλώθηκε.

```
void f() {
        int k = 0;
        System.out.println(k);
        {
                int i = 1;
        }
        System.out.println(i);
}
```

Προσέξτε πως σε αυτό το παράδειγμα έχουμε εμφωλιασμένα (nested) μπλοκ, δηλαδή έχουμε το μπλοκ της f και μέσα σε αυτό, ένα εσωτερικό μπλοκ μέσα στο οποίο δηλώνεται η i. Η εμβέλεια επομένως της i περιορίζεται στο εσωτερικό μπλοκ. Γενικά, η εμβέλεια των τοπικών μεταβλητών περιορίζεται μέσα στο μπλοκ που δηλώθηκαν. Αυτός είναι ένας λόγος στον οποίο οφείλεται ο χαρακτηρισμός τους ως τοπικές. Ένας δεύτερος λόγος σχετίζεται με τον χρόνο ζωής τους.

Χρόνος ζωής: Ο χρόνος ζωής (variable life time) μιας μεταβλητής είναι το διάστημα από τη δημιουργία της μεταβλητής με τη δέσμευση της κατάλληλης μνήμης μέχρι την καταστροφή της με την αποδέσμευση της μνήμης.

Για παράδειγμα, κατά την εκτέλεση της πρώτης γραμμής της f, δεσμεύεται χώρος στη μνήμη και έχουμε την έναρξη της ζωής της μεταβλητής. Με το τέλος εκτέλεσης της f(), ο χώρος για την k αποδεσμεύεται και έχουμε τη λήξη της ζωής της. Στο παράδειγμα αυτό η εμβέλεια μοιάζει να είναι παρόμοια έννοια με τον χρόνο ζωής. Ωστόσο, εμβέλεια και χρόνος ζωής συνιστούν δύο εντελώς διαφορετικές έννοιες. Η μεν εμβέλεια ορίζεται στο πλαίσιο του χρόνου μεταγλώττισης (Compile time), ο δε χρόνος ζωής ορίζεται στο πλαίσιο του χρόνου εκτέλεσης (Run time). Στην ενότητα 11.5 διαφοροποιείται πληρέστερα η εμβέλεια από τον χρόνο ζωής.

Η Java δεν επιτρέπει τη δήλωση μεταβλητής σε εσωτερικό μπλοκ με αναγνωριστικό που έχει χρησιμοποιηθεί σε αντίστοιχο εξωτερικό. Για παράδειγμα, ο κώδικας

```
void f() {
        int k = 0;
        {
                int k = 1;
        }
}
```

θα παράξει λάθος μεταγλώττισης.
Ας σημειωθεί ότι δήλωση και αρχικοποίηση μιας ή περισσότερων μεταβλητών μπορεί να γίνει στην ίδια γραμμή κώδικα, π.χ.

```
int k=0;
int k=1, z=2;
int k=2, z;
```

Στην πρώτη περίπτωση δηλώνουμε και αρχικοποιούμε την k, στη δεύτερη δηλώνουμε και αρχικοποιούμε τις k και z και στην τρίτη δηλώνουμε τις k και z, αρχικοποιούμε όμως μόνο την k.

3.1.2 Τοπικές Σταθερές

Σε πολλές περιπτώσεις χρειαζόμαστε μεταβλητές που η τιμή τους δεν μπορεί να αλλάξει μετά την αρχική εκχώρηση. Οι μεταβλητές αυτού του τύπου ονομάζονται σταθερές (constant variables). Μια τοπική μεταβλητή μετατρέπεται σε σταθερά αν κατά τη δήλωσή της χρησιμοποιηθεί η δεσμευμένη λέξη (reserved word) final. Για παράδειγμα,

```
final int K=1;
```

Ο κώδικας αυτός μέσα σε ένα μπλοκ δηλώνει μια τοπική σταθερά. Σε μια τοπική σταθερά μπορεί να εκχωρηθεί τιμή αυστηρά μία φορά, είτε μαζί με τη δήλωσή της είτε αργότερα. Επομένως, ο κώδικας

```
final int K;
K=1;
```

είναι έγκυρος.
Αντίθετα, ο κώδικας

```
final int K=1;
K=2;
```

παράγει λάθος μεταγλώττισης.

Όπως φαίνεται από τα παραδείγματα αυτής της ενότητας, οι συμβάσεις ονοματολογίας για τις σταθερές διαφέρουν από τις αντίστοιχες συμβάσεις για μεταβλητές. Οι σταθερές ονομάζονται αποκλειστικά με χρήση κεφαλαίων χαρακτήρων.

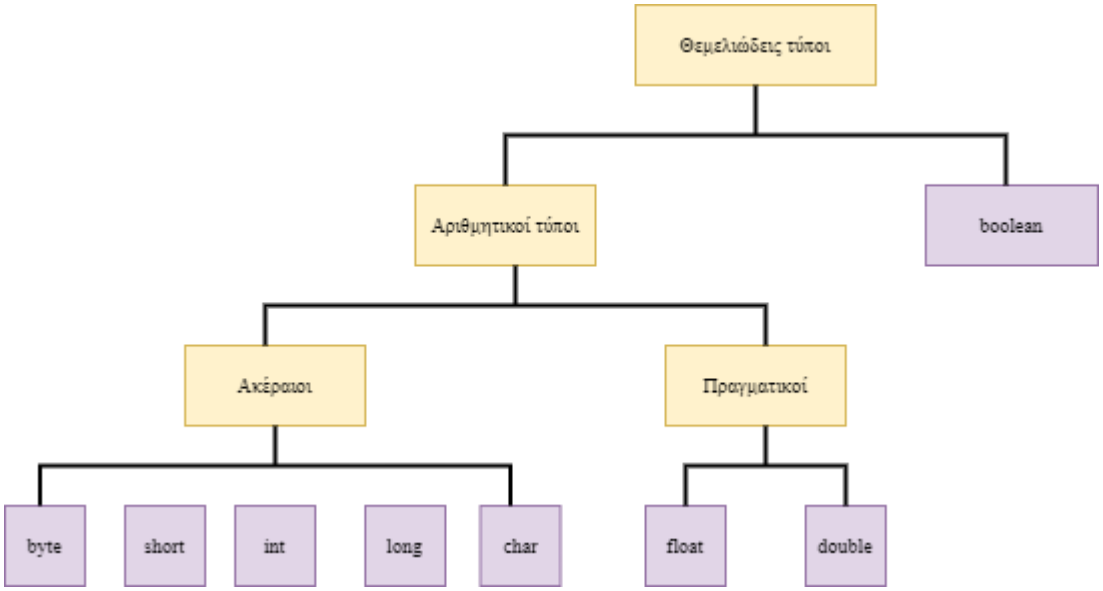
3.2 Θεμελιώδεις τύποι δεδομένων

Όπως εξηγήσαμε στην προηγούμενη ενότητα, τα προγράμματα διαχειρίζονται τα δεδομένα τους με τη βοήθεια των μεταβλητών. Τα δεδομένα είναι διάφορων τύπων, π.χ. ακέραιοι αριθμοί, πραγματικοί, χαρακτήρες, κλπ. Αντίστοιχα, και οι μεταβλητές που χρησιμοποιούνται τυποποιούνται ως ακέραιες, πραγματικές, μεταβλητές χαρακτήρων, κλπ. Η Java θέτει αυστηρούς κανόνες σχετικά με τον προσδιορισμό τύπου μιας μεταβλητής. Πιο συγκεκριμένα, απαιτεί ο προσδιορισμός τύπου κάθε μεταβλητής να είναι δεδομένος κατά τον χρόνο μεταγλώττισης και να παραμένει αμετάβλητος για όλη τη ζωή της μεταβλητής. Γλώσσες με παρόμοια συμπεριφορά χαρακτηρίζονται ως γλώσσες ισχυρού τύπου (Strong typed languages), σε αντίθεση με τις γλώσσες που δεν θέτουν αυτές τις απαιτήσεις και χαρακτηρίζονται ως γλώσσες αδύναμου τύπου (Weak typed languages).

Ένα επίσης σημαντικό χαρακτηριστικό της Java είναι η πρώιμη διασύνδεση (early binding). Όπως αναφέραμε μια μεταβλητή μπορεί να φιλοξενεί μια ακέραιη τιμή ή έναν χαρακτήρα. Και στις δύο περιπτώσεις όμως τα δεδομένα στη μνήμη αποθηκεύονται ως μια σειρά από δυαδικά ψηφία. Για να είναι δυνατή η ορθή ερμηνεία αυτών των δεδομένων είναι αναγκαίο να γνωρίζουμε τον τύπο της μεταβλητής. Με άλλα λόγια, όταν προσπελαύνουμε μια θέση μνήμης, δηλαδή μια σειρά από bits, πρέπει να είναι γνωστό αν η σειρά αυτή αναπαριστά έναν ακέραιο ή έναν χαρακτήρα. Η Java χαρακτηρίζει κάθε μεταβλητή με συγκεκριμένο τύπο, γνωστό κατά τον χρόνο μεταγλώττισης και σταθερό καθόλη την εκτέλεση του προγράμματος. Έτσι όταν προσπελαύνει μια μεταβλητή γνωρίζοντας τη διεύθυνσή της στη μνήμη και τον τύπο της μπορεί άμεσα να την αξιοποιήσει. Στον αντίποδα, οι γλώσσες καθυστερημένης διασύνδεσης (late binding), επιτρέπουν κατά τη διάρκεια εκτέλεσης του προγράμματος την εκχώρηση στην ίδια μεταβλητή δεδομένων ποικίλων τύπων. Για παράδειγμα, η Javascript, επιτρέπει την εκχώρηση σε μεταβλητή μιας ακέραιας τιμής και παρακάτω στον ίδιο κώδικα την εκχώρηση μιας συμβολοσειράς. Επομένως, ο τύπος της μεταβλητής δεν μπορεί να καθοριστεί παρά μόνο κατά τον χρόνο εκτέλεσης. Έτσι όταν η Javascript προσπελαύνει μια μεταβλητή θα πρέπει να υπολογίσει

ποιος είναι ο τύπος της σε ακριβώς εκείνο το σημείο της εκτέλεσης. Όπως είναι φανερό, η πρώιμη διασύνδεση βελτιώνει την ταχύτητα εκτέλεσης.

Οι μεταβλητές επομένως στην Java έχουν προκαθορισμένο τύπο. Οι διαθέσιμοι τύποι είναι πάρα πολλοί. Επιπλέον, ο χρήστης έχει τη δυνατότητα να ορίσει και δικούς του τύπους (User Defined Types). Σε αυτήν την ενότητα όμως παρουσιάζουμε ειδικά τους θεμελιώδεις τύπους (primitive types) της Java. Οι θεμελιώδεις τύποι είναι οκτώ [1], είναι προκαθορισμένοι (predefined) και τα ονόματά τους αποτελούν δεσμευμένες λέξεις (reserved words). Πλήρη λίστα με τις λέξεις κλειδιά της Java μπορείτε να βρείτε στο παράρτημα 2. Όλοι οι υπόλοιποι τύποι προκύπτουν από συνδυασμό ή επανορισμό των θεμελιωδών τύπων. Στο σχήμα 3.1, παρουσιάζεται η κατηγοριοποίηση των θεμελιωδών τύπων.



***Σχήμα 3.1**: Κατηγοριοποίηση των θεμελιωδών τύπων*

byte Ο τύπος byte είναι προσημασμένος ακέραιος μήκους οκτώ δυαδικών ψηφίων (bits). Αν η τιμή μιας μεταβλητής τύπου byte είναι θετική, τότε αναπαρίσταται στη μνήμη κωδικοποιημένη με τη μορφή πρόσημο και μέτρο (sign and magnitude), δηλαδή το πρώτο bit έχει την τιμή 0 για να δείξει πως ο αριθμός είναι θετικός και τα υπόλοιπα bits διατηρούν την τιμή του αριθμού εκφρασμένη στο δυαδικό σύστημα. Αν η τιμή μιας μεταβλητής τύπου byte είναι αρνητική, αυτή αναπαρίσταται στη μνήμη με τη μορφή συμπληρώματος ως προς 2 (two's complement) (παράρτημα 1). Σε αυτήν την περίπτωση το πρώτο bit έχει την τιμή 1 που δείχνει πως ο αριθμός είναι αρνητικός. Επομένως, τόσο για τους θετικούς όσο και για τους αρνητικούς, εξαιρώντας το πρώτο bit που ονομάζεται και bit πρόσημου, διαπιστώνουμε πως για την αναπαράσταση της τιμής απομένουν 7 bits. Συνεπώς, αναπαρίστανται συνολικά 27 αρνητικές τιμές και 27 μη αρνητικές τιμές. Πιο συγκεκριμένα, αναπαρίστανται οι αρνητικές τιμές -1..-27 και οι μη αρνητικές τιμές 0..27-1. Προσέξτε πως η μικρότερη αρνητική τιμή, δηλαδή το -27, είναι κατ' απόλυτη τιμή μεγαλύτερη κατά μια μονάδα από τη μεγαλύτερη μη αρνητική, δηλαδή το 27-1. Αυτό συμβαίνει γιατί στις μη αρνητικές τιμές συμπεριλαμβάνεται και το 0.

Η Java για κάθε θεμελιώδη τύπο διαθέτει και μια κλάση (Class) που παρέχει λειτουργίες σχετικές με τον θεμελιώδη τύπο. Την έννοια της κλάσης θα αναλύσουμε με λεπτομέρεια κατά τη μελέτη του αντικειμενοστρεφούς μοντέλου. Για την ώρα, θα θεωρούμε την κλάση ως ένα σύνθετο τύπο ο οποίος, εκτός από δεδομένα, ενσωματώνει και λειτουργίες (συναρτήσεις). Σε γενικές γραμμές, θεμελιώδεις τύποι και αντίστοιχες κλάσεις έχουν την ίδια ονομασία με μόνη διαφορά ότι το όνομα της κλάσης ξεκινάει με κεφαλαίο ενώ του τύπου με πεζό. Έτσι ο τύπος byte συνοδεύεται από την κλάση Byte.

```
static void byteDemo() {
        System.out.println("Size of byte is " + Byte.SIZE + " bits");
        byte bMin = Byte.MIN_VALUE, bMax = Byte.MAX_VALUE, bLiteral = 123;
        System.out.println("bMin equals to " + bMin);
        System.out.println("bMax equals to " + bMax);
        System.out.println("bLiteral equals to " + bLiteral);
}
```

***Κώδικας 3.1**: Παράδειγμα χρήσης μεταβλητής τύπου byte*

Στον κώδικα 3.1, καταρχάς, εμφανίζουμε το μέγεθος του τύπου (Byte.SIZE) μετρημένο σε δυαδικά ψηφία (bits). Μετά δηλώνουμε τις μεταβλητές bMin, bMax και bLiteral. Την πρώτη αρχικοποιούμε στην ελάχιστη τιμή του τύπου (Byte.MIN_VALUE), τη δεύτερη στη μέγιστη τιμή του τύπου (Byte.MAX_VALUE) και την τρίτη σε μια ενδιάμεση τιμή. Την ελάχιστη και μέγιστη τιμή μας παρέχει η κλάση Byte που τις περιλαμβάνει ως σταθερές μεταβλητές (variable constants). Για την ενδιάμεση τιμή χρησιμοποιούμε μια κυριολεκτική σταθερά (literal). Στη συνέχεια στέλνουμε στη συσκευή εξόδου, δηλαδή στην τυπική περίπτωση στην οθόνη, το περιεχόμενο των bMin, bMax και bLiteral.
Παρατίθεται η έξοδος του κώδικα 3.1.

```
Size of byte is 8 bits
bMin equals to -128
bMax equals to 127
bLiteral equals to 123
```

short Ο τύπος short είναι προσημασμένος ακέραιος. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Το μήκος του όμως είναι 16 δυαδικών ψηφίων. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου short είναι 0..215-1 και -1..-215.
Στον κώδικα 3.2, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου short.

```
static void shortDemo() { //Κώδικας 3.2
        short s = Short.MIN_VALUE, s1 = Short.MAX_VALUE, s2 = 399;
        System.out.println("short s equals to " + s);
        System.out.println("short s1 equals to " + s1);
        System.out.println("short s2 equals to " + s2);
        System.out.println("Size of short is " + Short.SIZE + " bits");
}
```

***Κώδικας 3.2**: Παράδειγμα χρήσης μεταβλητών τύπου short*

Ακολουθεί η έξοδος του κώδικα 3.2

```
short s equals to -32768
short s1 equals to 32767
short s2 equals to 399
Size of short is 16 bits
```

int Ο τύπος int είναι προσημασμένος ακέραιος μήκους 32 δυαδικών ψηφίων. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου int είναι 0..231-1 και -1..-231.
Στον κώδικα 3.3, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου int.

```
    static void intDemo() { //Κώδικας 3.3
        int iMin = Integer.MIN_VALUE,
                iMax = Integer.MAX_VALUE, iLiteral = 43127;
        System.out.println("int iMin equals to " + iMin);
        System.out.println("int iMax equals to " + iMax);
        System.out.println("int iLiteral equals to " + iLiteral);
        System.out.println("Size of int is " + Integer.SIZE + " bits");
    }
```

***Κώδικας 3.3**: Δήλωση και αρχικοποίηση μεταβλητών τύπου int*

Ακολουθεί η έξοδος του κώδικα 3.3.

```
int iMin equals to -2147483648
```

```
int iMax equals to 2147483647
int iLiteral equals to 43127
Size of int is 32 bits
```

Προσέξτε πως κατ' εξαίρεση, το όνομα της κλάσης που μας παρέχει πληροφορίες σχετικά με τον τύπο int, δεν είναι Int αλλά Integer.

long Ο τύπος long είναι προσημασμένος ακέραιος μήκους 64 δυαδικών ψηφίων. Η κωδικοποίησή του είναι ανάλογη με αυτήν του τύπου byte, δηλαδή πρόσημο και μέτρο για τους θετικούς και συμπλήρωμα ως προς 2 για τους αρνητικούς. Επομένως, οι τιμές που δέχονται οι μεταβλητές τύπου long είναι 0..263-1 και -1..-263. Στον κώδικα 3.4, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου long.

```
static void longDemo() { //Κώδικας 3.4
    long lMin = Long.MIN_VALUE,
            lMax = Long.MAX_VALUE, lLiteral = 2_147_483_699L;
    System.out.println("long lMin equals to " + lMin);
    System.out.println("long lMax equals to " + lMax);
    System.out.println("long lLiteral equals to " + lLiteral);
    System.out.println("Size of long is " + Long.SIZE + " bits");
}
```

***Κώδικας 3.4**: Δήλωση και αρχικοποίηση μεταβλητών τύπου long*

Ακολουθεί η έξοδος του κώδικα 3.4.

```
long lMin equals to -9223372036854775808
long lMax equals to 9223372036854775807
long lLiteral equals to 2147483699
Size of long is 64
```

Προσέξτε πως στην αρχικοποίηση της lLiteral, η κυριολεκτική σταθερά ακολουθείται από τον χαρακτήρα L. Οι ακέραιες κυριολεκτικές σταθερές που δεν ακολουθούνται από το L έχουν τύπο int. Αν θέλουμε να χρησιμοποιήσουμε κυριολεκτικές σταθερές με τύπο long, θα πρέπει αυτές να ακολουθούνται από το L. Στον κώδικα 3.4, αν αφαιρέσουμε το L, ο μεταγλωττιστής θα παράξει error, καθώς η τιμή 2147483699 είναι μεγαλύτερη από τη μέγιστη τιμή τύπου int. Ωστόσο σταθερές τύπου int (μέσα στα όρια του τύπου int) μπορούν να εκχωρηθούν σε μεταβλητές τύπου long, καθώς η Java κάνει αυτόματες μετατροπές μεταξύ συμβατών τύπων. Εναλλακτικά, αντί για κεφαλαίο L μπορεί να χρησιμοποιηθεί το πεζό l. Ωστόσο, συνίσταται η χρήση του κεφαλαίου καθώς το πεζό διακρίνεται με δυσκολία από τον αριθμό 1.

Προσέξτε επίσης πως ανάμεσα στα ψηφία έχουμε τοποθετήσει χαρακτήρες '_' (underscore). Από την έκδοση 7 και μετά, η Java επιτρέπει οποιονδήποτε αριθμό από underscores σε αριθμητικές σταθερές ώστε να διευκολύνεται η αναγνωσιμότητα των προγραμμάτων. Ωστόσο, δεν επιτρέπεται ο χαρακτήρας underscore, στην αρχή ή στο τέλος ενός αριθμού.

float Ο τύπος float χρησιμοποιείται για διαχείριση πραγματικών αριθμών. Έχει μήκος 32 δυαδικά ψηφία. Η κωδικοποίηση στη μνήμη των τιμών αυτού του τύπου ακολουθεί ένα πρότυπο γνωστό ως IEEE 754 και αναπτύχθηκε από το Ινστιτούτο Ηλεκτρολόγων και Ηλεκτρονικών Μηχανικών (Institute of Electrical and Electronics Engineers), που εδρεύει στη Νέα Υόρκη και στο οποίο συνήθως αναφερόμαστε με την αγγλική συντομογραφία IEEE. Η ανάλυση του προτύπου IEEE 754 είναι έξω από τα όρια αυτού του εγχειριδίου. Ωστόσο, είναι χρήσιμο να γνωρίζει κανείς πως οι πραγματικοί αριθμοί αναπαρίστανται σε εκθετική μορφή (Exponential Notation) και εκφράζονται με επιστημονικό συμβολισμό (scientific notation), mxEn, όπου m ένας πραγματικός αριθμός, n ακέραιος και E=10. Οι float αριθμοί χαρακτηρίζονται ως αριθμοί κινητής υποδιαστολής απλής ακρίβειας (single-precision floating point numbers).
Στον κώδικα 3.5, παρατίθεται παράδειγμα δήλωσης και αρχικοποίησης μεταβλητών τύπου float.

```
static void floatDemo() { //Κώδικας 3.5
        float fMin = Float.MIN_VALUE, fMax = Float.MAX_VALUE, fLit = 3.15f;
        System.out.println("float fMin equals to " + fMin);
```